EX27 · CLASS_17_VECTORS · 03 / GUÍA DEL EJERCICIO

EX27: Producto vectorial y momentos

Primero resolvemos álgebra de vectores; después calculamos momentos de fuerzas. Tú escribes el programa.

Ruta de aprendizaje
01 Teoría matemática → 02 Referencia Python → 03 Ejercicio → 04 Pseudocódigo → Programación.

Requisitos previos: EX25 y EX26, arrays NumPy, norma 2 y producto escalar.

1. Objetivos y condiciones

Resolver productos vectoriales 3D, comprobar su dirección y magnitud, y aplicarlos a momentos de fuerzas respecto a un punto de referencia.

Requisitos: un solo script, `numpy` y funciones ya explicadas. No necesitas JSON, `pytest`, clases ni módulos nuevos. No se exige crear funciones propias.

2. Parte A — Ejemplo algebraico de referencia

Comprueba a mano las relaciones de los vectores de la base cartesiana. Este ejemplo está resuelto para servirte de modelo.

$$\mathbf{a} = (1, 0, 0), \quad \mathbf{b} = (0, 1, 0)$$
$$\mathbf{a} \times \mathbf{b} = (0, 0, 1)$$
$$\mathbf{b} \times \mathbf{a} = (0, 0, -1)$$

Ambos resultados son vectores. Tienen magnitud 1, pero sentidos opuestos. Verifica estos valores con `np.cross`.

3. Parte A — Tu problema algebraico

Ahora aplica el mismo procedimiento a dos vectores de tres componentes que no están alineados con los ejes. Esta parte debes programarla tú.

$$\mathbf{u} = (2, -1, 3), \quad \mathbf{v} = (1, 4, -2)$$

1. Crea los dos arrays NumPy con tres componentes.

2. Calcula los productos **u** × **v** y **v** × **u**.

3. Calcula la norma 2 del primer producto vectorial.

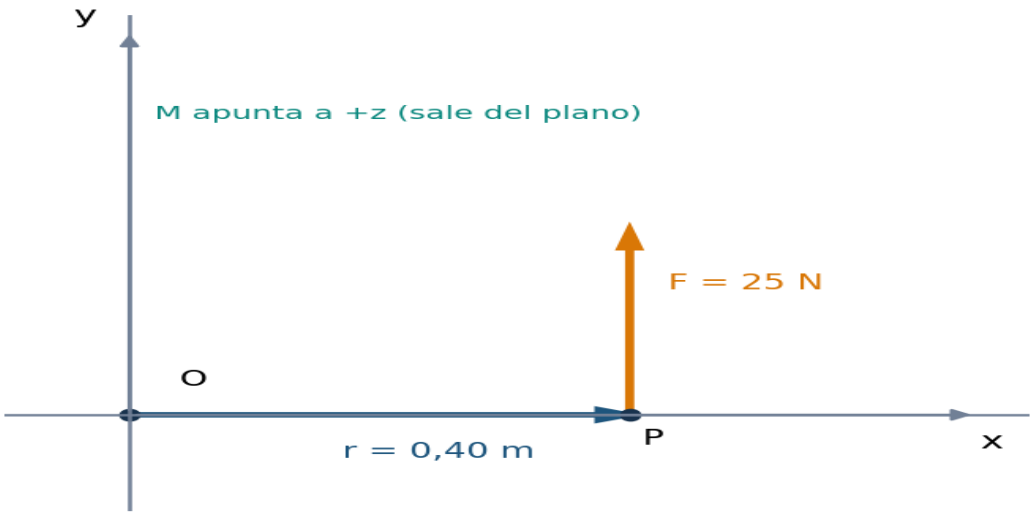
4. Comprueba que el resultado es perpendicular a los dos vectores originales mediante el producto escalar.

5. Imprime los resultados y compáralos con los valores de control.

Comprobación	Resultado de control
u × v	(−10, 7, 9)
v × u	(10, −7, −9)
Norma de u × v	√230 ≈ 15.1658
u · (u × v)	0
v · (u × v)	0

4. Parte B — Momento de una fuerza en 2D

Una fuerza de 25 N hacia +y actúa en el extremo de un brazo horizontal de longitud 0.40 m. Queremos el momento respecto al origen O. Aunque la geometría sea 2D, expresamos los vectores con 3 componentes para obtener la dirección del momento.



Esquema del enunciado. El vector de posición va desde O al punto de aplicación de la fuerza.

$$\mathbf{r}_{OP} = (0.40, 0, 0)\ \text{m}, \quad \mathbf{F} = (0, 25, 0)\ \text{N}$$

$$\mathbf{M}_O = \mathbf{r}_{OP} \times \mathbf{F}$$

1. Representa ambos vectores en NumPy.

2. Calcula el momento vectorial respetando el orden de los argumentos.

3. Calcula la magnitud del momento.

4. Explica si la dirección es +z o −z, y por qué.

> **Comprobación B**
> Momento vectorial: (0, 0, 10) N·m. Magnitud: 10 N·m. Dirección: +z, saliendo del plano xy.

5. Parte C — Caso 3D autónomo

Un segundo punto de aplicación se encuentra fuera de los ejes principales y recibe una fuerza tridimensional. No necesitas nuevos métodos; aplica lo aprendido.

$$\mathbf{r} = (0.30, 0.20, 0)\ \text{m}, \quad \mathbf{F} = (10, -5, 20)\ \text{N}$$

1. Calcula el vector momento respecto al origen.

2. Obtén su norma y presenta el resultado en N·m.

3. Realiza una comprobación manual de una componente, explicando el signo.

> **Comprobación C**
> Vector momento: (4, −6, −3.5) N·m. Norma aproximada: 8.016 N·m.

6. Requisitos de implementación y carpetas

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors/
            ├── EX25_vectors_and_forces.py
            ├── EX26_dot_product.py
            ├── EX27_cross_product.py   <--- NUEVO
            └── docs/
                ├── ES/
                └── EN/
```

Programa únicamente en `EX27_cross_product.py`. Separa A, B y C mediante comentarios breves. Puedes escribir el pseudocódigo primero en papel; después traslada los pasos a Python.

7. Preguntas de interpretación

Responde brevemente, en comentarios o al compartir el resultado:

- ¿Por qué el producto vectorial devuelve un vector y no un escalar?
- ¿Qué cambia si invertimos el orden de los vectores?
- ¿Por qué en un problema de fuerzas 2D aparece un momento en z?
- ¿Por qué las unidades del momento son N·m?

8. Ayuda de código — consulta al final

Las siguientes líneas muestran la sintaxis, pero **no resuelven** los apartados del ejercicio. Decide tú qué variables necesitas, en qué orden y qué comprobaciones vas a imprimir.

```
import numpy as np

vector = np.array([1.0, 0.0, 0.0])
other = np.array([0.0, 1.0, 0.0])

normal = np.cross(vector, other)
size = np.linalg.norm(normal)
check = np.dot(vector, normal)
```